nara-kosensai

システム構成・仕様書

対象リポジトリ: `nara-kosensai/manifest`(旧 `config`)、`fumikun/2026kosensai`
最終更新: 2026-08-29

目次

第 1 章 概要 2
第 2 章 アーキテクチャ 4
2.1 リクエスト経路 4
2.2 namespace 構成 6
2.3 Ingress ルーティング(旧 nginx からの移植) 7
2.3.1 Cloudflare Tunnel 側の ingress 設定 8
第 3 章 ネットワーク・セキュリティ 9
3.1 公開ポート 9
3.2 TLS / mTLS 9
3.3 RBAC 10
第 4 章 シークレット管理 11
第 5 章 CI/CD パイプライン 14
第 6 章 監視 16
第 7 章 障害と設計判断 18
7.1 nftables ルールのインターフェーススコープ(2026-08-29) 18
7.2 VPS 再起動によるホスト名変化と k3s ノード名の固定(2026-08-28) 18
7.3 push 型 CI から ArgoCD(GitOps)への移行(2026-09-02) 19
第 8 章 リファレンス 20

第 1 章　概要

nara-kosensai は、単一 VPS 上のシングルノード k3s クラスタで運用されている。旧構成(VPS ネイティブの nginx と複数の docker-compose)から全面移行したもので、外部通信は原則 Cloudflare Tunnel 経由のみとし、クラス タはノードの公開インターフェースに極力ポートをバインドしない。唯一の例外が本番アプリ (`nara-kosensai.com` / `www.nara-kosensai.com`)で、Cloudflare の Origin Authentication(mTLS)を使い、Tunnel を経由せず直接オリジンに到達させる。

このクラスタの設定は 4 つのリポジトリに分散して管理する。特定サービスに紐づかない共有プラット フォーム設定・クラスタブートストラップは本リポジトリ(`nara-kosensai/manifest`)に、アプリ/コンテ ンツ固有の設定はそれぞれのサービスのリポジトリに置く方針である(表 1.1)。

リポジトリ	内容
`nara-kosensai/manifest` (本リポジトリ)	クラスタブートストラップ(`k3s/`)、特定サービスに紐づかない共有プラットフォーム設定 (`k8s/`：cloudflared, Grafana/Loki/Prometheus, 過 去 年 度 ア ー カ イ ブ , storybook, kosensai-form-preview のデプロイ定義)、この仕様書。CI は `main` ブランチへの push で発火
`fumikun/2026kosensai`	当 年 ア プ リ 本 体 (backend/frontend/db/pgadmin/website)の Dockerfile・k8s マニフェスト (`infra/k8s/base/`, `overlays/{prod,test}`)・CI/CD。`prod`/`test` ブランチへの push でそれぞれ本番/検証環境にデプロイ
`nara-kosensai/www`	過去年度(2017–2025)アーカイブサイトの中身と、そのイメージをビルドする Dockerfile・CI。デプロイ先マニフェスト(`k8s/base/web-static/archive-*.yaml`)は 本リポジトリ側が保持する
`fumikun/kosensai-form-preview`	高専祭掲載情報プレビューツール本体・CI。デプロイ先マニフェストは本リポジトリ側 (`k8s/base/web-static/form-preview-*.yaml`)

表 1.1：設定リポジトリの役割分担

リポジトリ名について：本リポジトリは GitHub 上で `nara-kosensai/config` から `nara-kosensai/manifest` へリネーム済み(2026 年)。リネーム後も GitHub は旧 URL (`.../config.git`)から新 URL への参照を自動転送するため、旧 URL を `origin` に設定した 既存クローンは fetch/push とも動作し続けるが、新規クローンや CI の `checkout` アクションは 新 URL(`nara-kosensai/manifest`)を使うのが望ましい。ドキュメント内では現在の正式名 `manifest` を主に用い、旧名 `config` は文脈上必要な場合のみ併記する。

第 2 章　アーキテクチャ

2.1　リクエスト経路

外部からのリクエストは、宛先ホストによって 2 つの異なる経路のいずれかを通る。両方とも最終的に同一の Traefik Pod(namespace: `kube-system`)へ到達するが、Traefik の中では別々の entrypoint として扱わ れ、互いに干渉しない(図 2.1 に示す)。

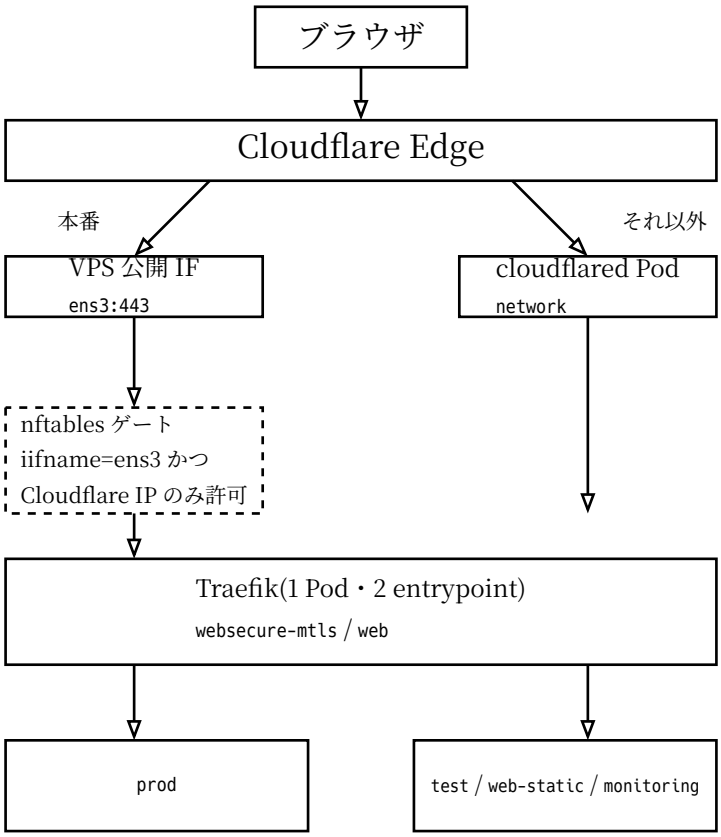


図 2.1：本番(mTLS Origin Auth)とそれ以外(Cloudflare Tunnel)、2 つの入口経路

図 2.1 の通り、`websecure-mtls` だけがホストポートに紐づいた entrypoint であり、`web` にはホストへの bind が一切ない。したがって `web` 側に残された IngressRoute は、たとえ nftables のフィルタやクラ イアント証明書の検証が何らかの理由で回避されたとしても、インターネットからは構造的に到達不可能であ る(§ 7 障害と設計判断も参照)。

段階	本番 — mTLS Origin Auth	それ以外 — Cloudflare Tunnel
対象ホスト	nara-kosensai.com / www.nara-kosensai.com	test.* / storybook.* / grafana.* / pgadmin*.* / info-check.*
Cloudflare エッジ	SSL/TLS: Full (strict) + Authenticated Origin Pulls	Cloudflare Tunnel(通常の DNS/SSL モード設定とは独立に動作)
VPS への入口	公開インターフェース ens3:443(hostPort)	cloudflared Pod(namespace: network)、VPS 側からの outbound 接続のみ
フィルタ	nftables raw prerouting：iifname == ens3 かつ tcp dport 443 かつ saddr ∉ Cloudflare IP → drop(無応答)	該当なし(公開ポートを一切持たないため到達手段自体が存在しない)
Traefik entrypoint	websecure-mtls・hostPort 443・サーバ証明書は Origin CA 証明書・TLSOption(RequireAndVerifyClientCert) でクライアント証明書を検証	web・ClusterIP :80 のみ・ホストへの bind なし
到達先 namespace	prod	test / web-static / monitoring

表 2.1：2 つの入口経路の対比

ソースコード 2.1 :Cloudflare IP allowlist(nftables、k3s/cloudflare-ip-allowlist.sh が生成。IPv4/IPv6 で独立したテーブルを持つ)

```nft
table ip cloudflare_mtls {
  set cf_ipv4 { type ipv4_addr; flags interval; elements = { ... } }
  chain prerouting {
    type filter hook prerouting priority raw; policy accept;
    iifname "ens3" tcp dport 443 ip saddr != @cf_ipv4 drop
  }
}
table ip6 cloudflare_mtls6 {
  set cf_ipv6 { type ipv6_addr; flags interval; elements = { ... } }
  chain prerouting {
    type filter hook prerouting priority raw; policy accept;
    iifname "ens3" tcp dport 443 ip6 saddr != @cf_ipv6 drop
  }
```

```
14 }
```

cloudflare-ip-allowlist.sh は冪等な全置換方式で動作する :新レンジ取得 → 一時ファイルに新 ruleset を組み立て → 既存の cloudflare_mtls / cloudflare_mtls6 テーブルを 削除 → nft -f で新 ruleset をアトミックに適用、という順序を取る。curl でのレンジ取得に失敗 した場合は set -euo pipefail によりテーブル削除前に異常終了するため、古いルールが残った フェイルセーフ側に倒れる。一方、テーブル削除後の nft -f 適用自体が失敗した場合は、次回の timer 実行(最大 24 時間+ランダム遅延)まで drop ルールが存在しない期間が生まれうる (既知の課題、 operations.md 参照)。

2.2 namespace 構成

namespace	代表ホスト	内容
prod	nara-kosensai.com / www	frontend・ backend・ db (StatefulSet)・ website・pgadmin
test	test.nara-kosensai.com	frontend・backend・db・website・pgadmin
web-static	storybook.* / info-check.*	www-archive(過 去 年 度 アー カ イ ブ)・ storybook・form-preview
monitoring	grafana.nara-kosensai.com	grafana・ loki・ prometheus・ node-exporter・kube-state-metrics
network	—	cloudflared(公開 Web 用 Tunnel)
argocd	argocd.nara-kosensai.com	ArgoCD 本体(GitOps コントローラ)
kube-system	—	Traefik(k3s 同梱)・CoreDNS

表 2.2：namespace 一覧

prod/test/monitoring/network/web-static の 実 際 の マ ニ フ ェ ス ト apply は ArgoCD (namespace: argocd)が git を直接 pull して行う。argocd 自体のインストールと、各 namespace の 作成・Secret 投入(ArgoCD の管理対象外)は引き続き各リポジトリの push-CI が担う (詳細は表 5.1 を参照)。web-static/prod/test の IngressRoute は、過去年度アーカイブ (web-static namespace)の Service を cross-namespace で参照する。単一テナント運用のクラスタ であるため、Traefik の providers.kubernetesCRD.allowCrossNamespace: true を有効にしている。

2.3 Ingress ルーティング(旧 nginx からの移植)

旧構成の nginx `default.conf` の server block が持っていたパス単位のルーティング・リダイレクト ロジックは、Traefik の IngressRoute/Middleware として `2026kosensai` リポジトリ側に移植した (`infra/k8s/overlays/{prod,test}/ingress/`)。nginx の longest-prefix-wins 相当を明示的な `priority`(数値が大きいほど高優先)で再現している(表 2.3)。

priority	マッチ条件(prod, Host=nara-kosensai.com)	Middleware	転送先
100	`Host(www.nara-kosensai.com)`	`www-to-apex`(恒久リダイレクト)	apex へ 301
90	`Path(/)` / `Path(/2026)` / `Path(/2026/app)`	`redirect-root` 等(末尾/付与)	301
80	`PathPrefix(/2026/app/)`	`security-headers`	frontend :3000
70	`PathPrefix(/2009..2016, /2021)`	`old-years-redirect`	学校公式サイトへ 301
60	`PathPrefix(/2026/)`	`security-headers` (test のみ `strip-2026-prefix` も併用)	website :80
10	`PathPrefix(/)`(catch-all)	`security-headers`, `archive-404`	www-archive :80 (namespace: web-static)

表 2.3：prod IngressRoute のルーティング優先度(test もほぼ同構成、mTLS ではなく Tunnel 経由)

補足:

- **`strip-2026-prefix`**：当年サイト(Astro)は `base: "/2026"`でビルドされ、生成 HTML 内の アセット参照は`/2026/...`前提。prod は旧 nginx の `alias` 相当の動作(内部的に

プレフィックス 除去)を website コンテナ側のビルド設定で吸収しているため Middleware 不要だが、test の website Service は素のリバースプロキシ転送のため明示的な `StripPrefix` が必要 (`infra/k8s/base/website/website-stripprefix.yaml`)。

- **`archive-404`**：`Host + PathPrefix(/)`の catch-all にヒットした結果を 404 として、 `web-static` namespace の `www-archive` Service が持つ`/2025/404.html` を errors Middleware 経由で返す(旧 nginx の `root /var/www/html/` + `/404.html` alias の再現)。
- **`security-headers`** Middleware(`k8s/base/ingress/security-headers.yaml`、`2026kosensai` リポジトリにも同一内容を複製配置)は、`X-Frame-Options`, `X-Content-Type-Options`, `Strict-Transport-Security` に加え、Google Tag Manager / Google Fonts / YouTube 埋め込みを許可する CSP を全ホスト共通で付与する。
- **pgAdmin**(`pgadmin.nara-kosensai.com` / `pgadmin-test.*`)は `web` entrypoint のみを使い (Tunnel 経由)、Traefik の Middleware ではなく Cloudflare Access (Tunnel の public hostname に紐づく認証ポリシー、ダッシュボード側で一度きり設定)で保護する。

2.3.1 Cloudflare Tunnel 側の ingress 設定

`web`entrypoint 行きの全ホストは、`cloudflared`(namespace: `network`)の ConfigMap(`k8s/base/network/cloudflared-configmap.yaml`)で一律 `http://traefik.kube-system.svc.cluster.local:80` へ転送されるよう設定してあり、ホスト単位の 実際の振り分けは前述の各 namespace の IngressRoute に委ねている。Tunnel の ingress ルールに現れる ホストは `test.*`, `storybook.*`, `info-check.*`, `pgadmin.*`, `pgadmin-test.*`, `grafana.*`で、末尾は `service: http_status:404` の catch-all で閉じる。本番ホスト (`nara-kosensai.com` / `www`)は、Origin Authentication 移行が完了した時点でこの ConfigMap の ingress エントリから削除済み(現在 Tunnel 経由で到達するのは `test`/`web-static`/ `monitoring` 配下のホストのみ)。

第 3 章 ネットワーク・セキュリティ

3.1 公開ポート

VPS の公開インターフェースに bind されるポートは 443/tcp のみ。80/tcp は Traefik の `web` entrypoint が ClusterIP のみで待ち受けており、ホストには一切 bind されない。

443/tcp へのアクセス制限は、VPS プロバイダ側とホスト側で役割が異なる。

(1) **VPS プロバイダのファイアウォール(セキュリティグループ)**：ポート単位の制限のみを行う (443/tcp と管理用 SSH 以外を閉じる)。送信元 IP アドレスによるフィルタリングは行っていない ―つまりこの層だけでは、Cloudflare 以外の任意の IP から 443/tcp へ到達できてしまう。

(2) **ホスト側 nftables**(`k3s/cloudflare-ip-allowlist.sh`、ソースコード 2.1)：送信元 IP アドレスの フィルタリングは、この層が唯一かつ全面的に担っている。Cloudflare が公開している IPv4/IPv6 レンジ(`https://www.cloudflare.com/ips-v4`, `ips-v6`)を、raw prerouting 段階でパケット の送信元インターフェース(`iifname`、デフォルトルートから動的に解決)と組み合わせて判定する。 マッチしない場合は TCP ハンドシェイクにも TLS アラートにも至らずサイレントに drop する。IPv4/IPv6 それぞれ独立したテーブル(`ip cloudflare_mtls` / `ip6 cloudflare_mtls6`)を持つ。systemd timer(`cloudflare-ip-allowlist.timer`)が日次でレンジを再取得し、`manifest` リポジトリへの push 時にも CI が都度再実行する。

したがって、Cloudflare IP アローリストの実体は完全に nftables 側にある。このスクリプトが失敗 した状態が長く続くと、VPS プロバイダ側のファイアウォールはそれを補ってくれない(運用マニュアル `operations.md` の「1. Cloudflare IP アローリストの更新不具合」を参照)。

3.2 TLS / mTLS

- **オリジンサーバ証明書**：Cloudflare Origin CA 証明書。SAN は `nara-kosensai.com` と `*.nara-kosensai.com` をカバー(有効期限 2041 年)。`2026kosensai` リポジトリの `production-vps` environment の GitHub Environment Secret(`CF_ORIGIN_CERT` / `CF_ORIGIN_KEY`)として保管し、CI が `prod` namespace に `cloudflare-origin-cert` (`kubernetes.io/tls`)Secret として投入する。
- **クライアント証明書検証**：Traefik の `TLSOption`(namespace: `network`、名前 `cloudflare-mtls`)が、Cloudflare が公開している Authenticated Origin Pulls 用 CA 証明書 (`k8s/base/network/cloudflare-aop-ca.yaml`、公開情報のため git 管理)に対して

RequireAndVerifyClientCert を要求する。Free プランのため Cloudflare 管理の zone-level 証明書を使用する(per-hostname のカスタムクライアント証明書は非対応)。

ソースコード 3.1：prod IngressRoute の TLS 設定(2026kosensai infra/k8s/overlays/prod/ingress/ingressroute.yaml)

```yaml
spec:
  entryPoints:
    - websecure-mtls
  tls:
    secretName: cloudflare-origin-cert
    options:
      name: cloudflare-mtls
      namespace: network
```

3.3 RBAC

Prometheus と kube-state-metrics は、それぞれ専用の ServiceAccount で動作し、read-only の get/list/watch のみを許可する ClusterRole を持つ(k8s/base/monitoring/prometheus-rbac.yaml、 kube-state-metrics.yaml)。既定の ServiceAccount には何の権限も付与しない。

kube-state-metrics 用 ClusterRole は list/watch のみだが、対象リソースには secrets が含まれる(Secret 自体の値ではなく、存在・メタデータを可視化するため標準的に必要とされる範囲)。書き込み権限は一切付与していない。Prometheus 用 ClusterRole はノード・Pod・Service・ Endpoints の get/list/watch に加え、kube-apiserver/kubelet/cAdvisor のメトリクス エンドポイントにアクセスするため ServiceAccount トークンを bearer token として使う (kubernetes_sd_configs による in-cluster discovery、TLS 検証は kubelet ジョブのみ insecure_skip_verify: true で緩めている。単一ノード・単一テナント運用のため許容)。

第 4 章　シークレット管理

Infisical(自己ホスト)は 2026-08-29 付けで完全に廃止した。現在の方針を表 4.1 に示す。

対象	保管場所	理由
DB / pgAdmin / CORS 等 (app-env)	manifest リポジトリ直下の prod.env / staging.env (平文コミット)	DB は ClusterIP 限定で外部到達不能
GHCR pull トークン	GitHub Environment Secret (production-vps / test-vps)	CI からのみ必要
Cloudflare Origin CA 証明書	GitHub Environment Secret (production-vps、環境限定)	本番専用の機微情報
Cloudflare Access クライアント資格情報	GitHub Actions Secret (管理用 SSH Tunnel 専用)	app-env とは無関係
Grafana 管理者パスワード等	k8s Secret (grafana-secrets) ブートストラップ時に手動投入	(gitignore 対象、循環依存を避けるため
ArgoCD 用 GitHub Deploy Key (公開 Web 用ポジトリ)	k8s Secret (argocd/repo-creds-manifest)	ArgoCD が manifest リポジトリを pull するため
GHCR pull 用 PAT (manifest リポジトリ)	k8s Secret (argocd/repo-creds-ghcr)	ArgoCD が Helm chart を pull するため
Cloudflare Tunnel 認証情報	k8s Secret (cloudflared/tunnel-credentials)	Tunnel 作成時に 1 度だけ生成される
Grafana Cloud 認証情報	k8s Secret (monitoring/grafana-cloud-credentials)	Alloy が remote_write するため
Argo Rollouts Dashboard 用 ServiceAccount トークン	k8s Secret (argo-rollouts/dashboard-token)	読み取り専用で kubectl 生成するため
Sentry DSN	app-env	公開前提の識別子
Cloudflare API Token	保存しない	一時的に使用するのみ
Cloudflare Zone ID / Account ID	保存しない	CLI 操作でのみ得られる

manifest リポジトリの CI が prod.env / staging.env を VPS の/home/github/app-env/に同期 し、2026kosensai リポジトリの CI がそのパスを直接--from-env-file で参照する。このため manifest 側の CI が 2026kosensai 側より先に一度動いている必要がある。同様に、web-static namespace の ghcr-pull Secret は manifest 側 CI が適用する k8s/overlays/shared が Deployment を作成する前提として存在している必要があり、これも一度きりの手動投入である。

prod/test namespace の ghcr-pull(2026kosensai アプリのイメージ pull 用)は 2026kosensai リポジトリの CI 自身が、GitHub Environment Secret(GHCR_PULL_TOKEN)から デプロイの都度 kubectl create secret ... --dry-run=client -o yaml ¦ kubectl apply -f -で 再生成する(こちらは一度きりの手動投入ではなく CI 管理)。トークン期限切れ時の症状・対処は operations.md の「3. GHCR pull トークンの期限切れ・権限エラー(403)」を参照。

第 5 章　CI/CD パイプライン

2026 年 9 月に push 型(CI が直接 `kubectl apply`)から ArgoCD による pull 型 GitOps へ移行した。 マニフェスト本体の apply はすべて ArgoCD が担い、各リポジトリの push-CI は「ArgoCD のスコープ外の もの(ArgoCD 自体のインストール、namespace 作成、Secret 投入)の同期」と「イメージ build/push + マニフェストのイメージタグ更新を git に commit&push」だけを行う。

リポジトリ / ブランチ	処理内容
`manifest` → `main`	k3s ブートストラップマニフェスト再配置 → Cloudflare IP allowlist 再実行 → `k8s/`を VPS へ rsync(ArgoCD が直接 git から pull するため apply はしない)→ ArgoCD 自体を `kubectl apply -k k8s/bootstrap/argocd`
`2026kosensai` → `prod`	backend/frontend/website/storybook 全イメージを無条件で build/push(path-filter は 使わない)→ `kustomize edit set image` で overlay のタグ更新 → そのまま `prod` ブランチに commit&push(`[skip ci]`)→ namespace 作成・GHCR pull トークン/Origin CA 証明書由来の Secret 投入
`2026kosensai` → `test`	変更検知(`dorny/paths-filter` で backend/frontend/website 差分を判定)→ 変更された コンポーネントのみ image build/push → 変更分のみ `kustomize edit set image` でタグ更新 → そのまま `test` ブランチに commit&push(`[skip ci]`)→ namespace 作成・GHCR pull トークン 由来の Secret 投入(pgAdmin 用 Cloudflare Access は別途手動設定)
`2026kosensai` → PR(任意ブランチ)	`pr-check.yml`：Node 側(pnpm + Turborepo)の `lint typecheck build`、Go 側 (`app/backend`)の `go build` / `go test`。デプロイは行わない

表 5.1：CI/CD パイプライン一覧

ArgoCD(namespace: argocd、argocd.nara-kosensai.com で Cloudflare Tunnel + Access 経由公開)は manifest リポジトリの k8s/argocd-apps/が定義する Application(App-of-apps、root-app)を 通じて、以下を git から直接 pull し automated(prune+selfHeal)で sync する。

Application	source repo / path	対象
platform-shared	manifest / k8s/overlays/shared	monitoring/network/ web-static/ingress
app-prod / app-test	2026kosensai(prod/ test) / infra/k8s/overlays/ {prod,test}/app-dynamic	backend/frontend/db/ pgadmin
website-prod / website-test	同上 / .../website	website
ingress-prod / ingress-test	同上 / .../ingress	IngressRoute/Middleware

表 5.2：ArgoCD Application 一覧

各リポジトリへの読み取りアクセスはリポジトリ単位の SSH Deploy Key で行う(argocdnamespace 内に argocd.argoproj.io/secret-type: repository ラベル付き Secret として登録、README.md 参照)。 CI は git push 後の ArgoCD の実際の sync/rollout 成否を待たない点に注意 (現状は ArgoCD UI/通知で監視する運用、CI 失敗としては検知しない)。

いずれも GitHub Actions から VPS へは、管理用の Cloudflare Access Tunnel(cloudflared access tcp) 経由で SSH 接続する。これは公開 Web 用の Tunnel(namespace network の cloudflared)とは別物で、VPS 側からの outbound 接続であるため、§3 の nftables allowlist とは無関係に動作する。SSH 接続に使う cloudflared 本体は、manifest/2026kosensai いずれのワークフローも、実行の都度 GitHub Releases API から最新バイナリをダウンロードし、リリースアセットの SHA-256 ダイジェストと突き合わせて検証してから使う(deploy-test.yml のみ 2026.7.2 にバージョン固定)。

2026kosensai の storybook イメージは deploy-prod.yml が build/push(タグは:latest と コミット SHA)まで行うが、実際に web-static namespace へ反映(kubectl apply)するのは manifest リポジトリ側の CI である。storybook は prod/test 共有の singleton であり、2026kosensai が管理する namespace(prod/test)には属さないため。manifest 側の CI は 毎回 deployment/storybook と deployment/form-preview を rollout restart してから :latest タグを再 pull させる(イメージダイジェストが変わってもタグ名は:latest のまま なので restart しないと Pod が更新されない)。

第 6 章　監視

Prometheus(namespace `monitoring`)は次をスクレイプする。

- `node-exporter`：ノードの CPU/メモリ/ディスク
- `traefik`：Traefik 自身のメトリクス(現状 9100 番ポートが未公開のため要修正、既知の課題)
- `kube-state-metrics`：Pod/Deployment/StatefulSet のオブジェクト状態 (Running/Restart 回数/Ready レプリカ数)
- `kubernetes-apiservers`：kube-apiserver のリクエストレイテンシ・エラー率
- `kubelet` / `kubernetes-cadvisor`：ノード・Pod 単位のリソース使用量

いずれも `kubernetes_sd_configs` による in-cluster discovery と、ServiceAccount トークン(`/var/run/secrets/kubernetes.io/serviceaccount/`)による認証を使う標準的な構成。Grafana の Prometheus/Loki データソースは自動プロビジョニングしておらず、初回のみ手動追加が必要 (`http://prometheus:9090`, `http://loki:3100`、いずれも `monitoring` namespace 内)。ログは Grafana Alloy(DaemonSet、`k3s/alloy-logging.yaml`)が全 namespace の Pod ログを Loki へ出荷する。

コンポーネント	Retain?	備考
`prometheus`(10Gi)	されない (`local-path`)	メトリクスは再収集可能なため失っても許容
`loki`(10Gi)	されない (`local-path`)	`retention_period: 72h`(=3 日)。ログも同様に許容
`grafana`(5Gi)	**される** (`local-path-retain`)	ダッシュボード定義・アノテーション等は再構築コストが高い

表 6.1：監視コンポーネントの PVC と reclaimPolicy(§5「シークレット管理」で扱う `app` 側の DB/backend も同じ `local-path-retain` を使う)

node-exporter は旧構成の `network_mode: host` 相当を、DaemonSet に `hostNetwork: true` / `hostPID: true` と`/proc`, `/sys`, `/`の hostPath マウントを組み合わせて再現している(単一ノードだが、将来ノードが増えても自動的に追随する構成)。Grafana は `GF_AUTH_GITHUB_ENABLED` による GitHub OAuth ログインを有効化しており、`GF_AUTH_GITHUB_ALLOWED_USERS` で許可した GitHub アカウントのみサインアップ・ログインできる(allow-list 外のアカウントは弾かれる)。管理者パスワードと GitHub OAuth のクライアントシークレットは `grafana-secrets`

Secret (表 4.1)からのみ注入し、それ以外の非機微な設定は Deployment マニフェストに直接 書く。

第 7 章　障害と設計判断

7.1　nftables ルールのインターフェーススコープ(2026-08-29)

Origin Authentication 導入時に追加した nftables ルールが、当初 `tcp dport 443` のみで判定してお り、送信元インターフェースを指定していなかった。クラスタ内部には `kubernetes.default.svc` (ClusterIP、`10.43.0.1:443`)を含む複数の ClusterIP がたまたま 443 番を使っており、このルールは Pod→API サーバー間の内部通信まで巻き込んで drop してしまった。

結果として `helm-install-traefik` ジョブ(Traefik の新しいエントリーポイント設定を適用するための Helm インストール)が API サーバーに到達できず、インストール失敗→アンインストール→再インストール のクラッシュループに陥り、Traefik 自体が消滅。本番環境が約 1 時間ダウンした(Cloudflare エラー 521/526)。

対処：ルールを `iifname`(ノードの実際の外部 NIC。ハードコードではなく `ip route show default` から都度動的に解決)でスコープした(ソースコード 2.1)。クラスタ内部のトラフィックは定義上この物理イン ターフェースを通らないため、どの ClusterIP が 443 番を使っていても衝突しない。

同時に、Traefik のコンテナポート・Service 公開ポートでも類似の衝突が発生していた(`websecure-mtls` を chart のデフォルト `websecure` と同じ 8443 番/443 番で定義していたため)。コンテ ナポートは 8444 番、Service 公開は `expose.default: false`(hostPort のみ、Service 経由の公開はそも そも不要)に変更して解消した。

7.2　VPS 再起動によるホスト名変化と k3s ノード名の固定(2026-08-28)

VPS プロバイダ側のメンテナンスと思われる OS 更新(カーネル `6.1.0-49-cloud-amd64` → `6.12.105+deb13-cloud-amd64`)を伴う再起動で、OS の静的ホスト名が `x162-43-87-201` から `kosensai` に変化した。k3s はデフォルトでノード名に OS ホスト名をそのまま使うため、同じ物理 マシンが**別のノード**として新規登録され、旧ノードオブジェクトは `NotReady` のまま残った。

このクラスタは `local-path`(ノードローカルストレージ)を使っており、既存の PersistentVolume はすべて `nodeAffinity` で旧ノード名を固定的に参照していた(この `nodeAffinity` は作成後 変更不可)。新ノード名はこの affinity にマッチしないため、PVC を使う全 Pod(`db`, `loki`, `grafana`, `prometheus` 等)が新ノードにスケジュールできず `Pending` のまま止まり、CI の `rollout status` 待ちがタイムアウトする形で表面化した。

対処：9 個の PV すべてを削除→再作成するのは本番データを巻き込むリスクが高いため、代わりに `/etc/systemd/system/k3s.service` の `ExecStart` に`--node-name x162-43-87-201` を追加し、**k3s 側のノード名を OS ホスト名から独立させて固定した**。以後 OS ホスト名が再び変わっても、k3s は常に同じノード名として登録されるため、既存 PV の `nodeAffinity` と食い違わない。

7.3 push 型 CI から ArgoCD(GitOps)への移行(2026-09-02)

デプロイを CI から直接 `kubectl apply` する push 型から、ArgoCD が git を直接 pull して apply する pull 型 (GitOps)へ移行した。狙いは主に 2 点：(1) VPS 上の実際の状態と git の内容が乖離した場合に `selfHeal` で自動的に検知・是正できるようにする、(2) 各リポジトリの CI から `rsync`+ `kubectl apply`+複数の `rollout status` 待ちという長い手続きを排除し、CI の責務を「イメージ build/push + マニフェストのタグ更新を git に反映する」ことだけに縮小する。

ArgoCD 自体は GitOps 管理の対象外とした(`k8s/bootstrap/argocd/`、push-CI が恒久的に適用)。理由は、ArgoCD の管理対象に ArgoCD 自身を含めると、ArgoCD が何らかの理由で壊れた場合に ArgoCD 自身を使った復旧ができなくなるため(chicken-and-egg 問題)。これは既存の `k3s/`ブートストラップ層(k3s 本体・Traefik 設定など)が push-CI 管理のまま残っている設計判断と同じ理由による。

Secret 類(`app-env`, `ghcr-pull`, `grafana-secrets`, `cloudflared-credentials`, `cloudflare-origin-cert`)はこれまで通り git にコミットせず kustomize の `resources` にも含めない 運用を維持した。ArgoCD の管理対象は各 kustomize base の resources に列挙されたものだけなので、これらの Secret は ArgoCD の `selfHeal` による prune 対象にもならず、CI が引き続き `kubectl create secret ... --dry-run=client -o yaml ¦ kubectl apply -f -`で投入する。

既知のトレードオフ：CI は `git push` 後、ArgoCD による実際の sync/rollout の成否を待たない (移行前は `kubectl rollout status` で CI ジョブ自体が失敗検知していた)。現状は ArgoCD UI/通知での 監視に委ねている。将来的に `argocd app wait` を Cloudflare Access Tunnel 経由で CI から呼ぶ形に 拡張し、CI での失敗検知を復元する余地がある(未実装)。

この事例は、シングルノード k3s でも「ノードの同一性」を OS ホスト名任せにすると壊れうる、という 運用上の教訓として設計判断に残す。応急処置・恒久対策の詳細な手順は `operations.md` 「2. VPS 再起動でホスト名が変わり k3s ノードが二重化する」を参照。

第 8 章　リファレンス

項目	値
Kubernetes ディストリビューション	k3s(シングルノード、`--disable=servicelb`)
k3s ノード名	`x162-43-87-201` に固定(`--node-name`、OS ホスト名から独立。§ 7 参照)
Ingress コントローラ	Traefik 3.7(k3s 同梱 Helm チャート)
CNI	flannel(k3s 標準)
ストレージ	local-path(既定、reclaimPolicy: Delete)、local-path-retain(reclaimPolicy: Retain、DB/backend/grafana 用)
公開ポート	443/tcp のみ(80/tcp は ClusterIP 限定で非公開)
Traefik `websecure-mtls`	containerPort 8444, hostPort 443, expose.default: false
Traefik `web`	containerPort 8000 相当 (chart 既定), ClusterIP:80 のみ
管理用 SSH	Cloudflare Access Tunnel(公開 Web 用 Tunnel とは別物、outbound-only)
Prometheus / Grafana / Loki	`prom/prometheus:v2.51.0` / `grafana/grafana:11.0.0` / `grafana/loki:3.0.0`
node-exporter / kube-state-metrics	`prom/node-exporter:v1.8.0` / `registry.k8s.io/kube-state-metrics:v2.13.0`
Loki 保持期間	`retention_period: 72h`(3 日)
関連リポジトリ	`nara-kosensai/manifest`(本リポジトリ、旧 `config`)、`fumikun/2026kosensai`、`nara-kosensai/www`、`fumikun/kosensai-form-preview`

表 8.1：設定値リファレンス